

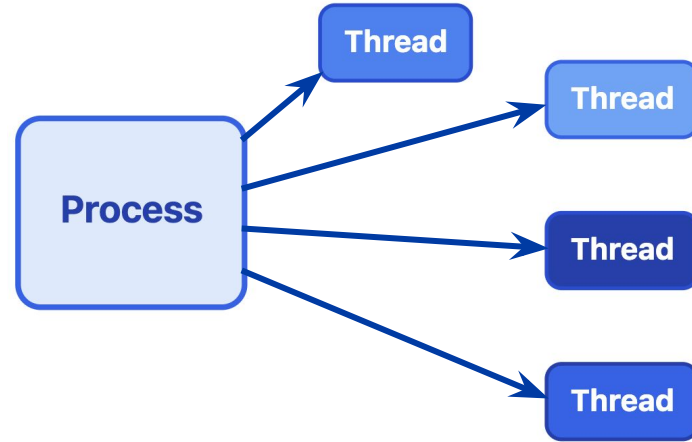
L2: Process & Threads

Department of Computer Science and Engineering



International Institute of Information Technology,
Bhubaneswar

Process



Utkalika Satapathy
utkalika@iiit-bh.ac.in

Readings / References

Reading key

G: Silberschatz, Galvin & Gagne, Operating System Concepts (use the available edition).

T: Tanenbaum & Bos, Modern Operating Systems (4th ed. or available edition).

S: William Stallings, Operating Systems: Internals and Design Principles (6th ed.).

P: Research paper or authoritative technical reference; assign selected sections rather than the entire paper.

Week 1 — Foundations

What is OS, Role of OS (user view and system view), Evolution of OS, History of OS

Galvin: Ch. 1

Tanenbaum: Ch. 1;

Stallings: Ch. 2

Week 2 — Foundations

Computer-system organization, I/O structure, Interrupts, Traps, Booting an OS, Types of OS, OS Structures, Different Computing Environments, OS Concepts, OS services, Introduction to popular operating systems like UNIX & Windows, System calls, OS Design Goals, APIs, Protection modes

Galvin: Ch. 1-2;

Tanenbaum: Ch. 1;

Stallings: Ch. 1-2

Week 3 — Processes & Threads

Process & Thread, Process concept, states, PCB, context switching, Process creation/termination; cooperating processes; IPC overview, Inter-Process Communication (IPC), IPC: shared memory, pipes, message passing, client-server communication, Threads, multithreading models, thread libraries and issues

Galvin Ch. 3-4

Tanenbaum Ch. 2;

Stallings: Ch. 3-4

Week 4 — Process Scheduling

Process Scheduling, Scheduling criteria; preemptive vs non-preemptive; dispatcher, FCFS, SJF/SRTF, priority and round-robin algorithms, Multilevel queue/feedback; starvation and aging; algorithm comparison, Multiprocessor, multicore and real-time scheduling overview

Galvin Ch. 5;

Tanenbaum Ch. 2;

Stallings: Ch. 9-10



Topics to be covered

- 01 - Process
- 02 - Process concept
- 03 - Process States
- 04 - PCB
- 05 - Context switching
- 06 - Process creation/termination
- 07 - Thread
- 08 - Multithreading models
- 09 - Thread libraries and issues

Process

OS for Processes

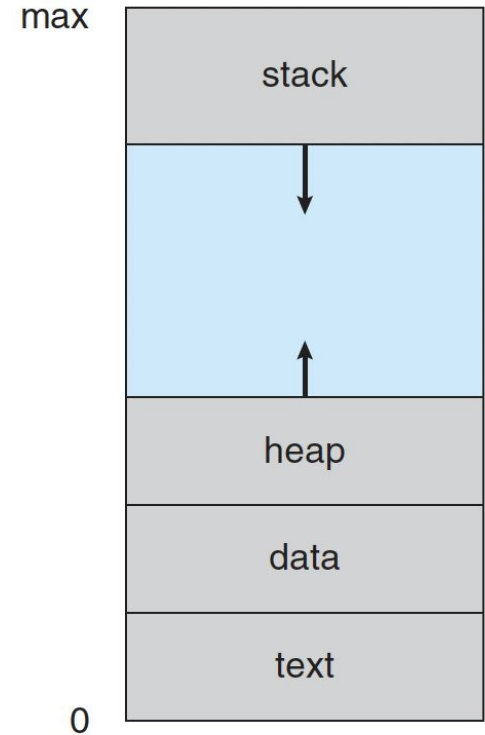
- Most requirements that the OS must meet can be expressed with reference to processes:
 - The OS must **interleave** the execution of multiple processes, to **maximize processor utilization** while providing **reasonable response time**.
 - The OS must **allocate resources** to processes in conformance with a **specific policy** (e.g., certain functions or applications are of higher priority) while at the same time avoiding deadlock.
 - The OS may be required to support **interprocess communication** and **user creation of processes**, both of which may aid in the structuring of applications.

Various Definition Processes

- An operating system executes a variety of programs:
 - Batch system – **jobs**
 - Time-shared systems – **user programs or tasks**
- Textbook uses the terms **job** and **process** almost interchangeably
- **Program** is **passive entity** stored on disk (executable file), **process** is **active**
 - Program becomes process when executable file loaded into memory
- Process is:
 - A program in execution
 - An instance of a program running on a computer
 - The entity that can be assigned to and executed on a processor
 - A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources

Process In Memory

- The memory layout of a process is typically divided into multiple sections:
 - Text section—the executable code
 - Data section—global variables
 - Heap section—memory that is dynamically allocated during program run time
 - Stack section—temporary data storage when invoking functions (such as function parameters, return addresses, and local variables)

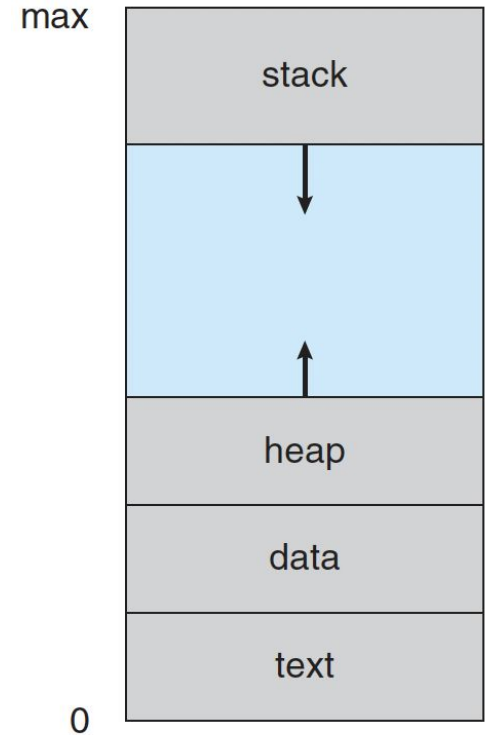


Source: G. Galvin

Fig 3.1 Layout of a process in memory

Process In Memory

- The sizes of the text and data sections are fixed, as their sizes do not change during program run time.
- However, the stack and heap sections can shrink and grow dynamically during program execution.
- Each time a function is called, an **activation record** containing function parameters, local variables, and the return address is **pushed onto the stack**; when control is returned from the function, the activation record is popped from the stack.
- Similarly, the heap will grow as memory is dynamically allocated, and will shrink when memory is returned to the system
- Although the stack and heap sections grow toward one another, the operating system must ensure they do not overlap one another.



Source: G. Galvin

Fig 3.1 Layout of a process in memory

Process Memory Map / Process Address Space

C Program

```
#include <stdio.h>
#include <stdlib.h>

int calls;           // Global data

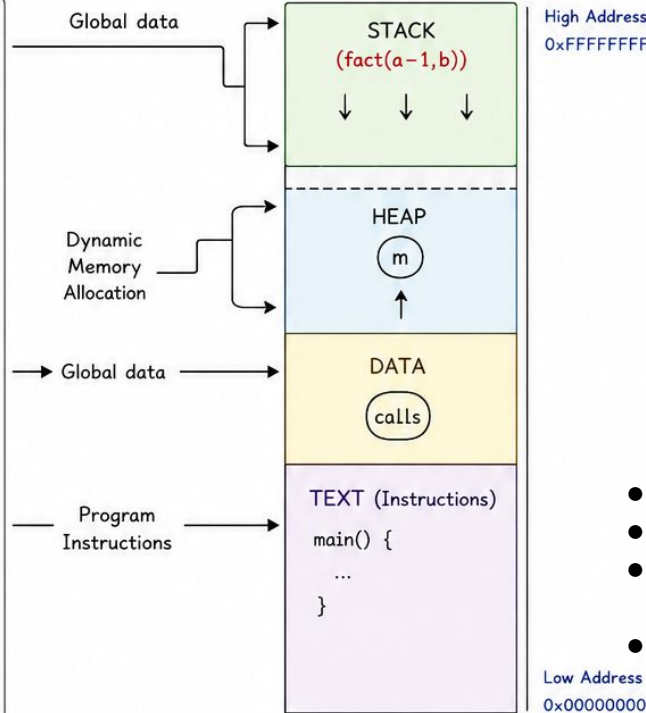
void fact (int a, int *b) {
    calls++;
    if (a == 1) return;
    *b = *b * a;
    fact (a-1, b);    // Function invocation
}

int main() {
    int n, *m;        // Local variable
    scanf("%d", &n);
    m = malloc(sizeof(int)); // Dynamic
                                // Memory Allocation

    *m = 1;
    fact(n, m);
    printf("fact(%d) is: %d\n", n, *m);
    free(m);
}
```

(Program)

Memory Map of a Process

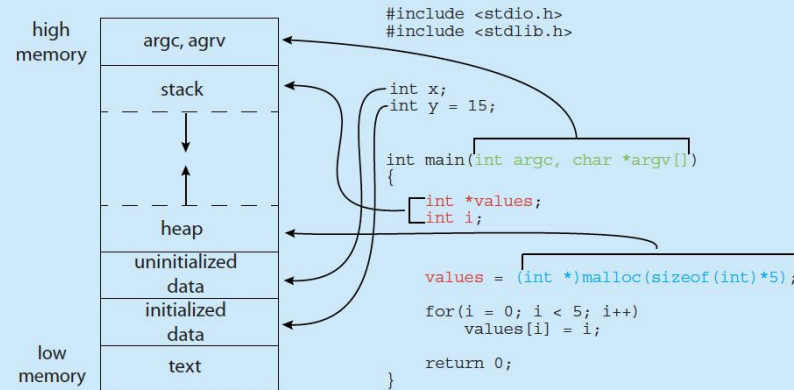


(Memory Map of the Process)

- The memory of a running program in DRAM has the following components
 - Compiled code (instructions)
 - Compile-time data (global/static variables)
 - Runtime data on stack (function arguments, local variables, ...)
 - Runtime data on heap (dynamically allocated memory via malloc, ...)
- All instructions and data are assigned memory addresses, based on their location in memory
- Main memory contains user programs + code/data of OS
- Stack grows downward (toward lower address)
- Heap grows upwards (towards higher address)
- Code(text), data, heap and stack together form the user space of a process
- Kernel space (OS) is same for all the process

Memory Layout of a C Program

- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the `argc` and `argv` parameters passed to the `main()` function.



Source: Galvin
Chapter 3

The GNU `size` command can be used to determine the size (in bytes) of some of these sections. Assuming the name of the executable file of the above C program is `memory`, the following is the output generated by entering the command `size memory`:

text	data	bss	dec	hex	filename
1158	284	8	1450	5aa	memory

The `data` field refers to uninitialized data, and `bss` refers to initialized data. (`bss` is a historical term referring to *block started by symbol*.) The `dec` and `hex` values are the sum of the three sections represented in decimal and hexadecimal, respectively.



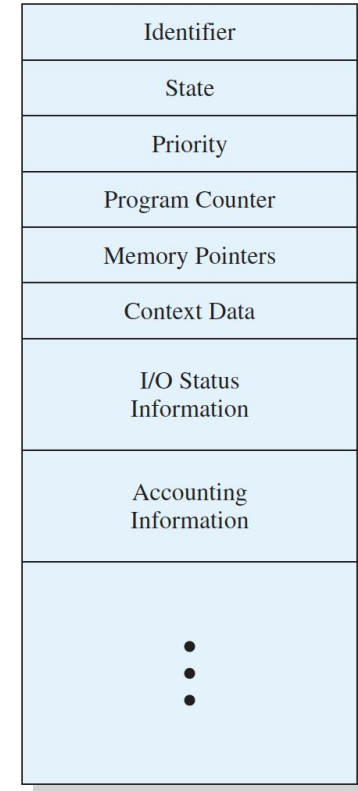
Process Control Block

- Two essential elements of a process are **program code** (which may be shared with other processes that are executing the same program) and a **set of data** associated with that code and we refer to this **executing entity as a process**.
- While the program is executing, this process can be uniquely characterized by a number of elements, including the following:
 - **Identifier**
 - **State**
 - **Priority**
 - **Program counter**
 - **Memory pointers**
 - **Context data**
 - **I/O status information**
 - **Accounting information**
- The information in the preceding list is stored in a **data structure**, typically called a **process control block** which is created and managed by the OS.



Process and Process Control Block

- **Identifier** – A unique identifier associated with this process, to distinguish it from all other processes.
- **State** – If the process is currently executing, it is in the running state. [The state may be new, ready, running, waiting, halted, and so on.]
- **Priority** – Priority level relative to other processes.
- **Program counter** – The address of the next instruction in the program to be executed.
- **Memory pointers** – Includes pointers to the program code and data associated with this process, plus any memory blocks shared with other processes
- **Context data** – These are data that are present in registers in the processor while the process is executing.
- **I/O status information:** Includes outstanding I/O requests, I/O devices (e.g., tape drives) assigned to this process, a list of files in use by the process, and so on.
- **Accounting information:** May include the amount of processor time and clock time used, time limits, account numbers, and so on.

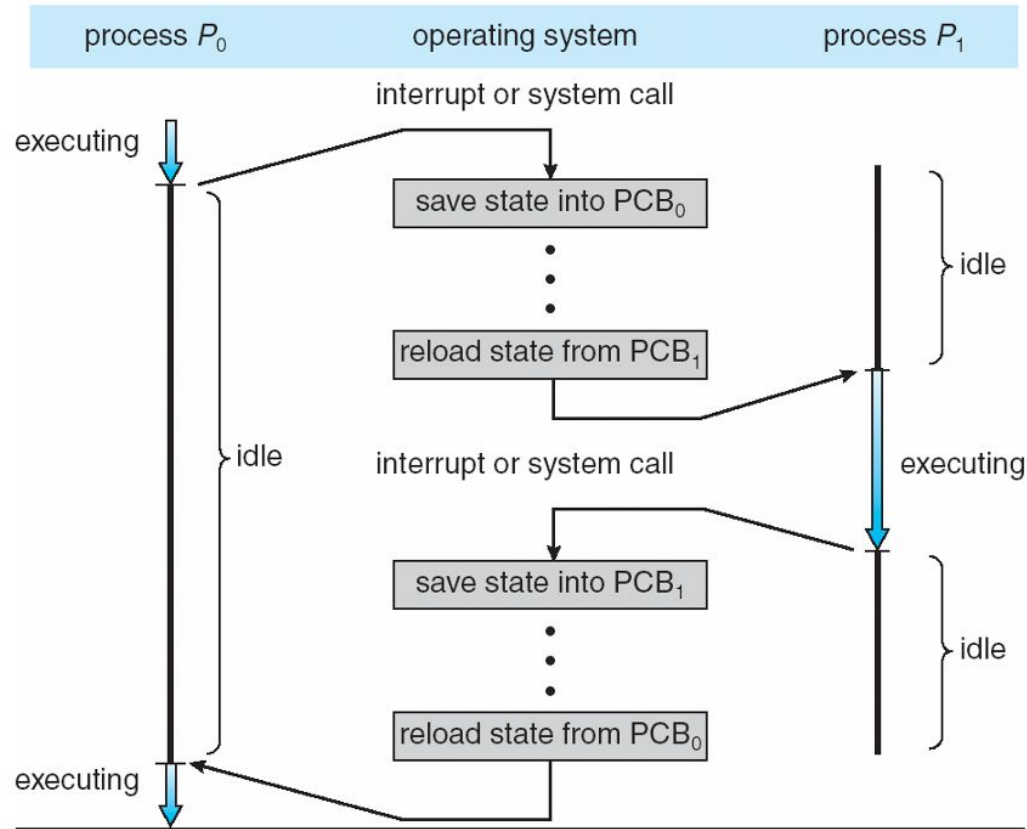


Source: W. Stallings Fig 3.1
Simplified Process Control Block₁₂

Why need PCB

- It contains sufficient information so that it is possible to interrupt a running process and later resume execution as if the interruption had not occurred.
- It enables the OS to support multiple processes and to provide for multiprocessing.
 - When a process is interrupted, the current values of the program counter and the processor registers (context data) are saved in the appropriate fields of the corresponding process control block, and the state of the process is changed to some other value, such as blocked or ready.
 - The OS is now free to put some other process in the running state.
 - The program counter and context data for this process are loaded into the processor registers and this process now begins to execute.

CPU Switch From Process to Process



Task Array / Task List

- Task Array / Task List - A data structure maintained by Linux Kernel
- This array contains pointers to the PCBs of all active process in the system
- PCBs are often linked together in different linked list or queues within the kernel
- PCB is the kernel representation of process allowing the OS to efficiently manage and control the process.
- Referred as “Task_Structure” or “Process_Descriptors”: data structure that serves as the central storage of information about processes.
- Whenever a new process is created then the memory is allocated for its “task_struct”

```
$ pahole -C task_struct vmlinux

struct task_struct {
    struct thread_info thread_info;          /*    0    8 */
    volatile long int      state;            /*    8    4 */
    void *                  stack;           /*   12    4 */

    ...

    /* --- cacheline 45 boundary (2880 bytes) --- */
    struct thread_struct thread __attribute__((__aligned__(64))); /* 2880 4288 */

    /* size: 7168, cachelines: 112, members: 155 */
    /* sum members: 7148, holes: 2, sum holes: 12 */
    /* sum bitfield members: 7 bits, bit holes: 2, sum bit holes: 57 bits */
    /* paddings: 1, sum paddings: 2 */
    /* forced alignments: 6, forced holes: 2, sum forced holes: 12 */
} __attribute__((__aligned__(64)));
```

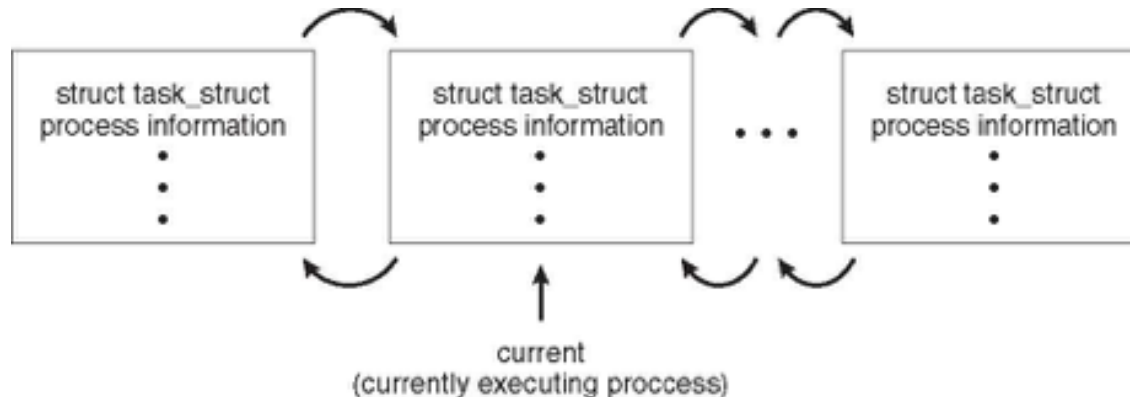
Source: [task_struct -Linux Page](#)



Process Representation in Linux

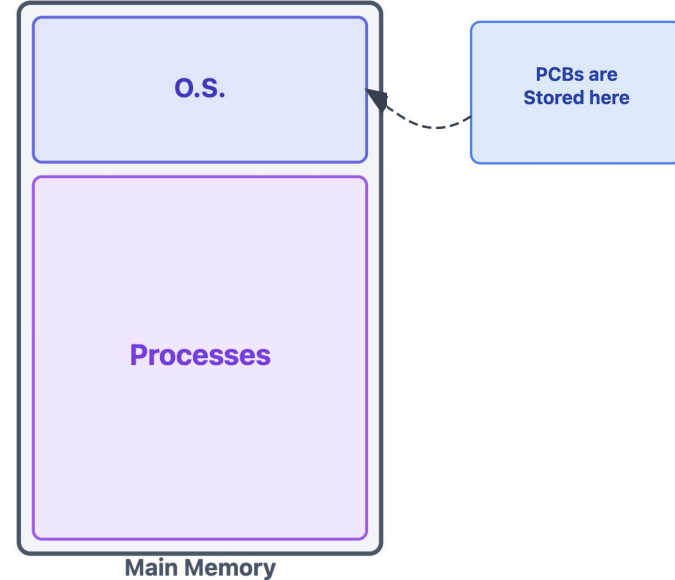
- Represented by the C structure **task_struct**

```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```



Where is PCB

- In Linux, the memory has 2 central region :
 - user-space and kernel-space
- User process run in an allocated user space that is separate from the kernel.
- Kernel memory space is reserved for the Linux Kernel and its data structure including PCB.
- PCBs are dynamically created in the kernel space at the process creation. When a new process starts, kernel allocates memory for its PCB.
- The unique identifier of each process is PID which serves as an index for the corresponding PCB.
- When a kernel need to access a particular PCB, it uses its PID to locate the PCB in memory.



Source: [task_struct - Linux Page](#)
[task_struct – Bootlin](#)

Role of PCB in Process Management

1. Process Scheduling
2. Context Switching
3. Resource Management
4. Signal Handling

Role of PCB in Process Management

1. Process Scheduling:

- On Linux the scheduler determines which process use the CPU
- How PCB helps:
 - i. **Process State:** Each PCB contains the info about the process state (RUNNING, WAITING, READY). The scheduler consults these states to determine which process to execute next
 - ii. **Priority and Scheduling Parameters:** This info helps the scheduler allocate CPU time fairly and efficiently based on the user-defined or OS-defined criteria
 - iii. **Process Queues:** Process in the Ready State are queued according to their priorities. These queues are maintained by the scheduler to determine the order in which processes are scheduled to run
 - iv. **Load Balancing:** In systems with multiple CPUs and cores, the PCB assist in load balancing. The scheduler may move processes across the CPUs to distribute the workload evenly. PCB track CPU affinity of each process.



Role of PCB in Process Management

2. Context Switching:

- Context Switching is the mechanism by which kernel saves the state of a running process, loads the state of another process and allows smooth transition between processes.
- How PCB helps:
 - i. **Save Process State:** When context switching occurs, the PCB of the currently running process saves its state to include CPU registers, program counters, and other relevant info. This vital info becomes the part of the PCB
 - ii. **Loading Process State:** The PCB of the next process to run is loaded and its saved state is restored from the PCB. This allows the process to continue execution from where it left off, ensuring the integrity of the its execution context.

Role of PCB in Process Management

3. Resource Management:

- Linux process requires various system resource such as memory, file descriptors and CPU time
- How PCB helps:
 - i. **Memory Management:** The PCB contains a reference to the process address space (`mm_struct`). This helps in memory allocation and deallocations ensuring that process don't interfere with each other's memory
 - ii. **File Descriptor Table:** The PCB maintains a file descriptor table (`files_struct`) that keeps track of the opened file and network connections. This table allows process to access files and sockets efficiently and ensure proper cleanup when processes terminates
 - iii. **Resource Limits:** PCB may include info about resource limits to impose on processes preventing overuse

Role of PCB in Process Management

4. Signal Handling:

- Linux processes can receive signals from kernel or other processes
- How PCB helps:
 - i. **Signal Handlers:** The PCB stores information about registered signal handlers for the process.
 - ii. **Pending Signals:** PCB may maintain a list of pending signals for a process. This list helps ensure that PCB does not lose these signals even if the process is not currently handling them

Process State

Process State

- We can characterize the behavior of an individual process by listing the sequence of instructions that execute for that process. Such a listing is referred to as a **trace of the process**
- We can characterize behavior of the processor by showing how the traces of the various processes are interleaved.
- The first 12 instructions executed in processes A and C are shown. Process B executes four instructions, and we assume that the fourth instruction invokes an I/O operation for which the process must wait.
- The **dispatcher program** that switches the processor from one process to another

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of Process A

(b) Trace of Process B

(c) Trace of Process C

5000 = Starting address of program of Process A

8000 = Starting address of program of Process B

12000 = Starting address of program of Process C

Source: W. Stallings

Fig 3.3 Traces of Processes of Figure 3.2



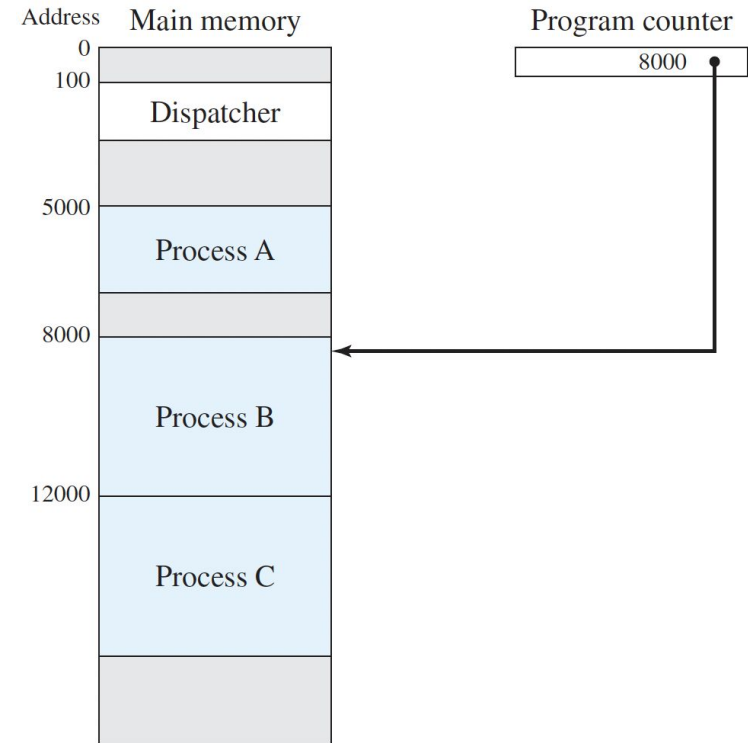
Process State

Source: W. Stallings
Fig 3.4
Combined
Trace of
Processes of
Figure 3.2
It shows the
interleaved
traces resulting
from the first
52 instruction
cycles

1	5000	27	12004
2	5001	28	12005
3	5002		Timeout
4	5003	29	100
5	5004	30	101
6	5005	31	102
	Timeout	32	103
7	100	33	104
8	101	34	105
9	102	35	5006
10	103	36	5007
11	104	37	5008
12	105	38	5009
13	8000	39	5010
14	8001	40	5011
15	8002		Timeout
16	8003	41	100
	I/O Request	42	101
17	100	43	102
18	101	44	103
19	102	45	104
20	103	46	105
21	104	47	12006
22	105	48	12007
23	12000	49	12008
24	12001	50	12009
25	12002	51	12010
26	12003	52	12011
			Timeout

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed



Source: W. Stallings
Fig 3.2 Snapshot of Example Execution (Fig 3.4) at
Instruction Cycle 13

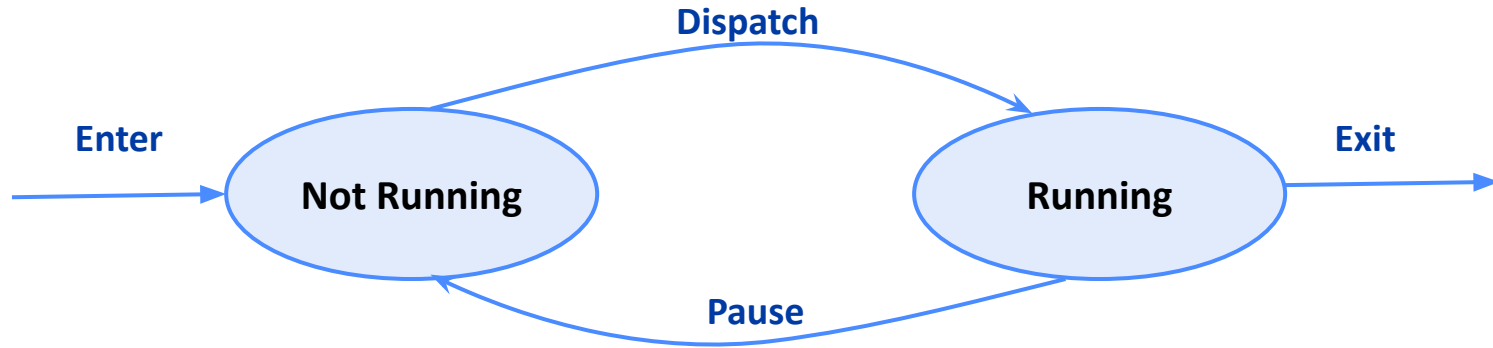
The same sequence of instructions is executed by the dispatcher in each instance because the same functionality of the dispatcher is being executed



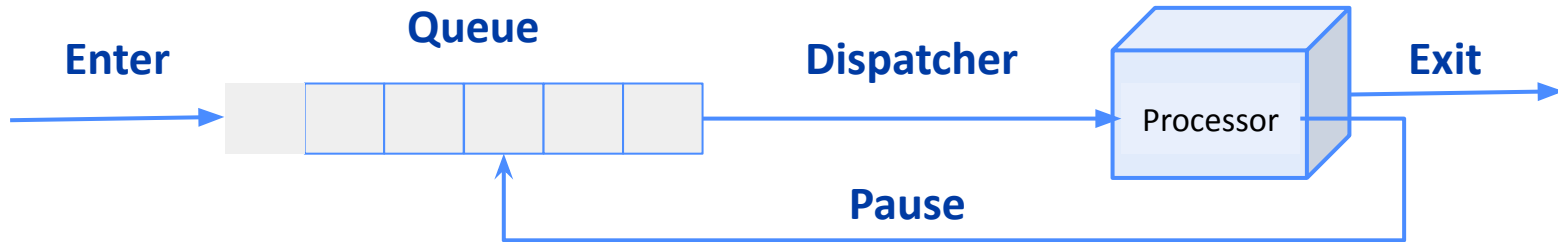
Two-State Process Model

- The operating system's principal responsibility is controlling the execution of processes.
- The simplest possible model by observing that, at any time, a process is either being executed by a processor or not
- In this model, a process may be in one of two states: **Running** or **Not Running**
- Processes that are not running must be kept in some sort of queue, waiting their turn to execute
- There is a single queue in which each entry is a pointer to the process control block of a particular process

Two-State Process Model



State Transition Diagram



Queuing Diagram – To describe the behavior of the dispatcher in terms of this queuing diagram.

The Creation of Processes

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive logon	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Source: W. Stallings

Table 3.1 Reasons for Process Creation

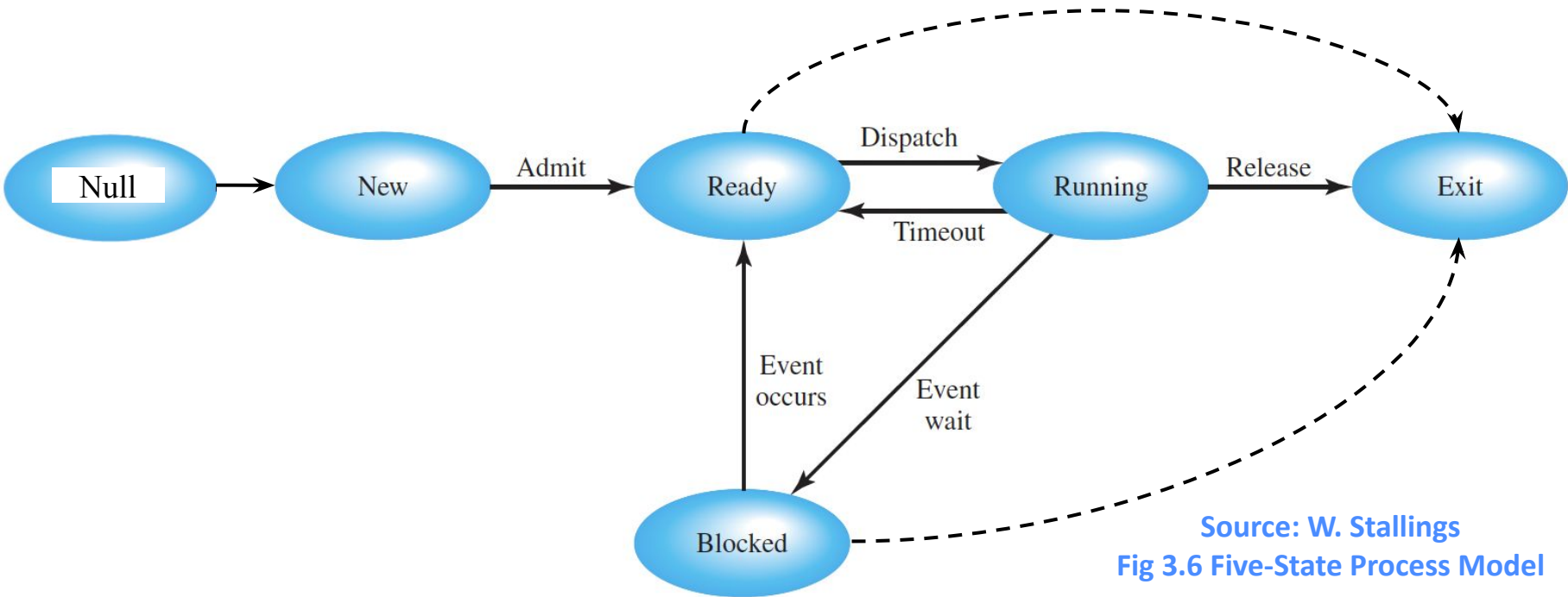
Note: When the OS creates a process at the explicit request of another process, the action is referred to as **process spawning**.

The Termination of Processes

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time (“wall clock time”), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (for example, if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Source: W. Stallings
Table 3.2 Reasons for Process Termination

Five-State Process Model



- The Not-Running state in Two-State Process Model is splitted into two states: Ready and Blocked.

Five-State Process Model

1. New:

- A process that has just been created but has not yet been admitted to the pool of executable processes by the OS.
- Typically, a new process has not yet been loaded into main memory, although its process control block has been created

2. Ready:

- A process that is prepared to execute when given the opportunity.

3. Running:

- The process that is currently being executed.

4. Blocked/Waiting:

- A process that cannot execute until some event occurs, such as the completion of an I/O operation

5. Exit:

- A process that has been released from the pool of executable processes by the OS, either because it halted or because it aborted for some reason.

State Transitions

1. Null → New:

- A new process is created to execute a program.
- This event occurs for any of the reasons listed the table for process creation.

2. New → Ready:

- The OS will move a process from the New state to the Ready state when it is prepared to take on an additional process.
- A program moves from new state to ready state after it is loaded from secondary memory into main memory and is ready for execution
- Most systems set some limit based on the number of existing processes or the amount of virtual memory committed to existing processes.
- This limit assures that there are not so many active processes as to degrade performance.

3. Ready → Running

- When it is time to select a process to run, the OS chooses one of the processes in the Ready state.
- This is the job of the scheduler or dispatcher.



State Transitions

4. Running → Exit:

- The currently running process is terminated by the OS if the process indicates that it has completed, or if it aborts. See Slide# 13

5. Running → Ready:

- When the running process has reached the maximum allowable time for uninterrupted execution

6. Running → Blocked:

- If a process requests something for which it must wait.
- For example, a process may request a service from the OS that the OS is not prepared to perform immediately.
- It can request a resource, such as a file or a shared section of virtual memory, that is not immediately available. Or the process may initiate an action, such as an I/O operation, that must be completed before the process can continue. When processes communicate with each other, a process may be blocked when it is waiting for another process to provide data or waiting for a message from another process.

State Transitions

7. Blocked → Ready:

- A process in the Blocked state is moved to the Ready state when the event for which it has been waiting occurs.

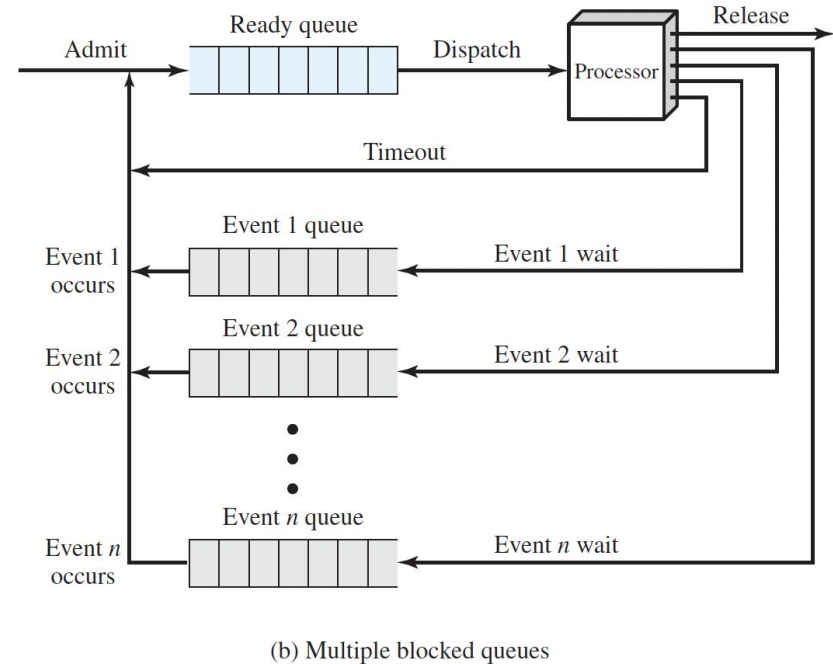
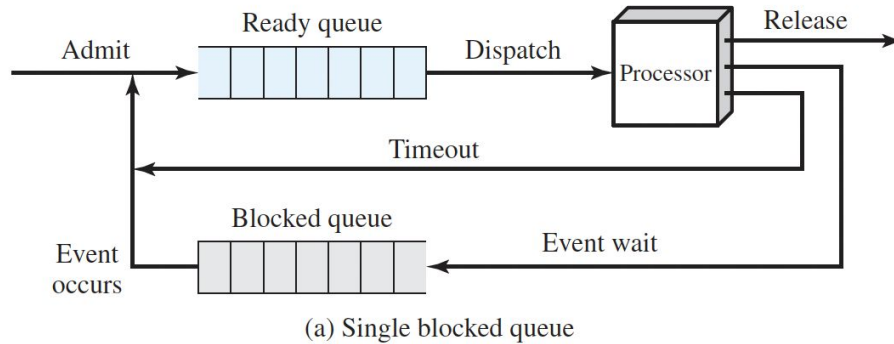
8. Ready → Exit (Not shown in the diagram):

- In some systems, a parent may terminate a child process at any time.
- Also, if a parent terminates, all child processes associated with that parent may be terminated.

9. Blocked → Exit

- In process-state terms, a blocked process normally cannot directly exit, because it is not currently executing. It must first become ready, get scheduled to run, and then terminate:

Queuing Model

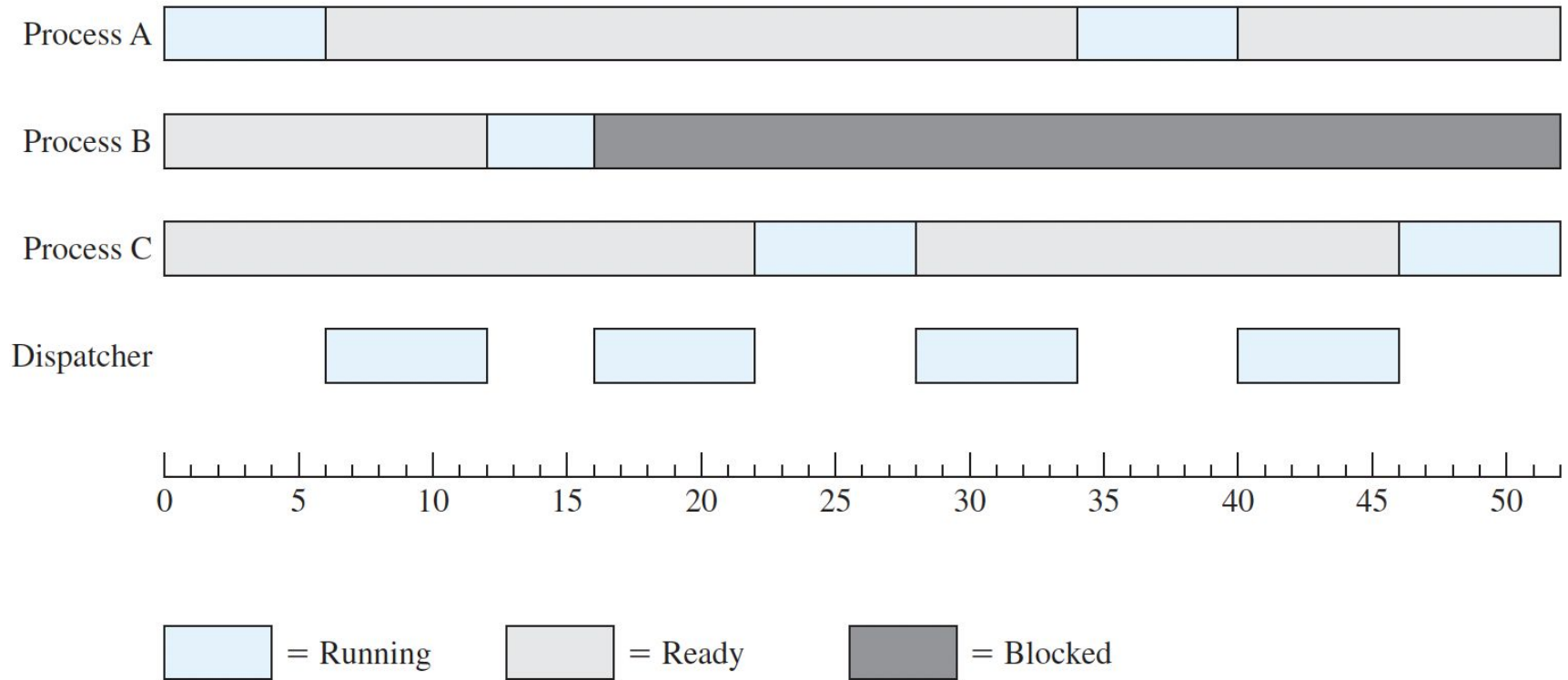


- As each process is admitted to the system, it is placed in the **Ready queue**.
- When it is time for the OS to choose another process to run, it selects one from the Ready queue based on some criteria.
- When a running process is removed from execution, it is either terminated or placed in the **Ready** or **Blocked queue**, depending on the circumstances.
- Finally, when an event occurs, any process in the **Blocked queue** that has been waiting on that event only is moved to the Ready queue.

Source: W. Stallings

Fig 3.8 Queuing Model for Figure 3.6

Process States for the Trace of Figure 3.4



Source: W. Stallings

Fig 3.7 Process States for the Trace of Figure 3.4

Suspended Processes (The Need for Swapping)

- Moving part or all of a process from main memory to disk.
- When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out onto disk into a **suspend queue**.
- Suspend queue: This is a queue of existing processes that have been temporarily kicked out of main memory, or suspended
- The OS then brings in another process from the suspend queue, or it honors a new-process request.
 - Execution then continues with the newly arrived process.

The Suspension of Process

1. Swapping

- The OS needs to release sufficient main memory to bring in a process that is ready to execute.

2. Other OS reason

- The OS may suspend a background or utility process or a process that is suspected of causing a problem

3. Interactive user request

- A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.

4. Timing

- A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.

5. Parent process request

- A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants

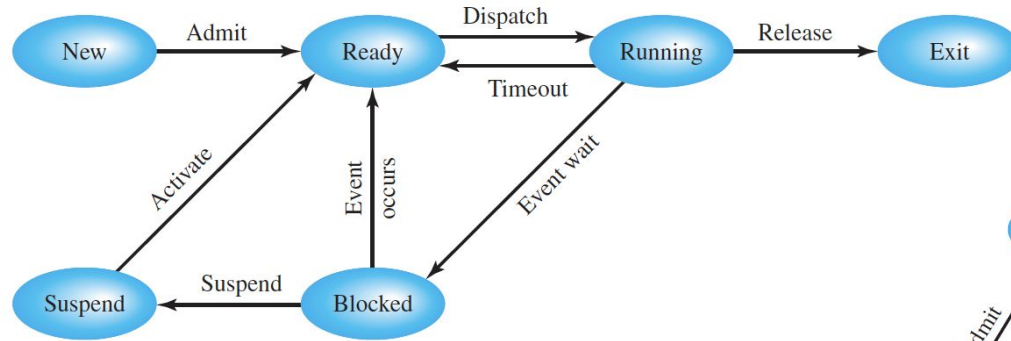
The Suspension of Processes

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

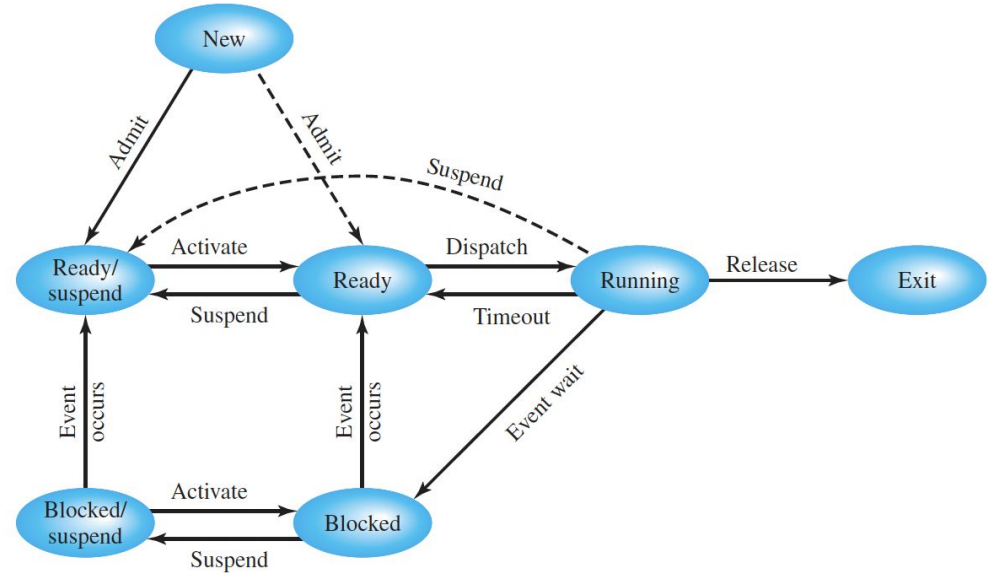
Source: W. Stallings

Table 3.3 Reasons for Process Suspension

Process State Transition Diagram with Suspend States



(a) With one suspend state



(b) With two suspend states

Source: W. Stallings

Fig 3.9 Process State Transition Diagram with Suspend States

Process Model with Suspend States

1. **New:**

- A process that has just been created but has not yet been admitted to the pool of executable processes by the OS.
- Typically, a new process has not yet been loaded into main memory, although its process control block has been created

2. **Ready:**

- The process is in main memory and available for execution.

3. **Blocked:**

- The process is in main memory and awaiting an event.

4. **Blocked/Suspend:**

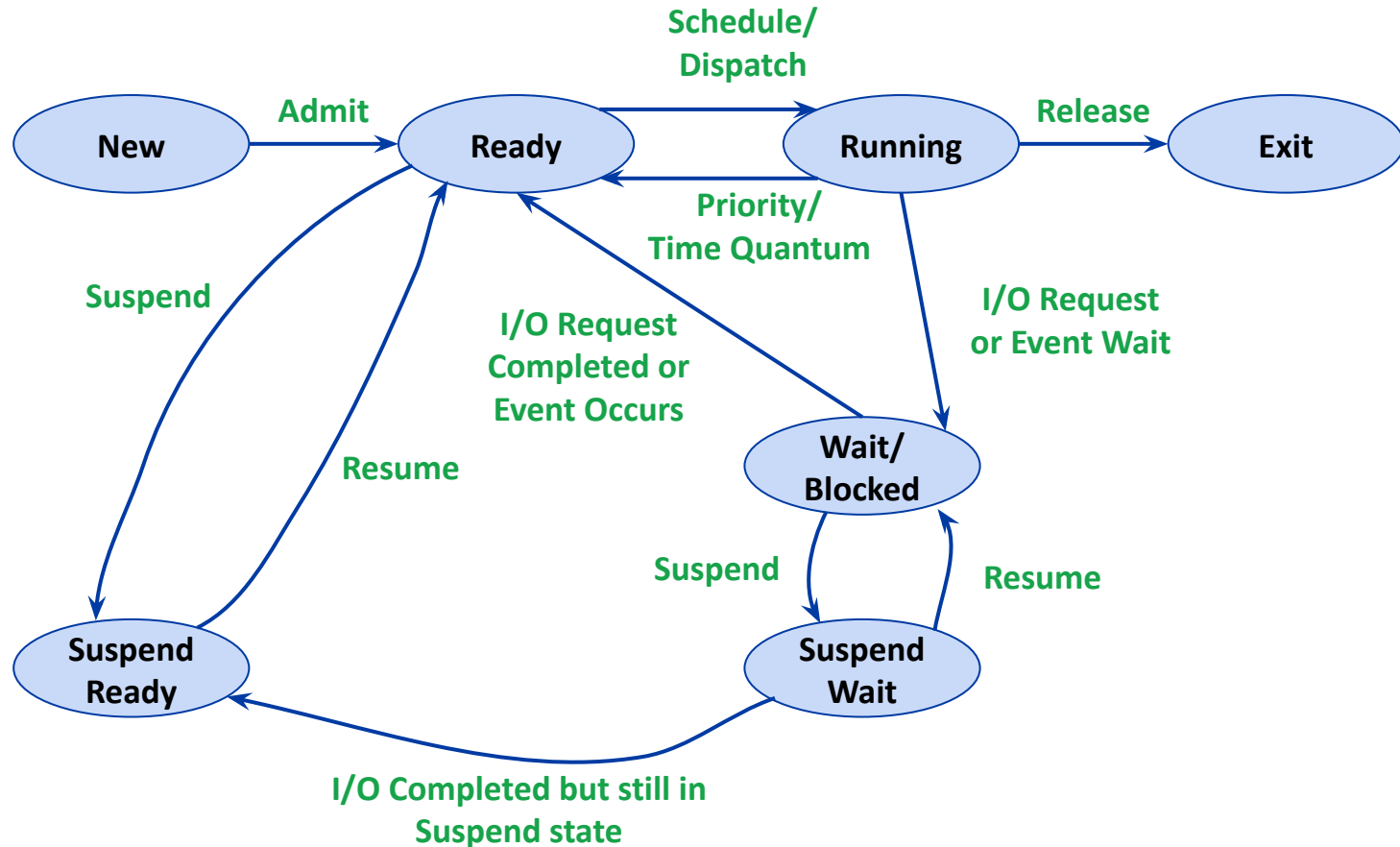
- The process is in secondary memory and awaiting an event.

5. **Ready/Suspend:**

- The process is in secondary memory but is available for execution as soon as it is loaded into main memory.



Process State Transition Diagram with Suspend States



Process State Transition Diagram with Suspend States

1. **Blocked** → **Blocked/Suspend**:

- If there are no ready processes, then at least one blocked process is swapped out to make room for another process that is not blocked.
- This transition can be made even if there are ready processes available, if the OS determines that the currently running process or a ready process that it would like to dispatch requires more main memory to maintain adequate performance.

2. **Blocked/Suspend** → **Ready/Suspend**:

- A process in the Blocked/Suspend state is moved to the Ready/Suspend state when the event for which it has been waiting occurs.
- Note that this requires that the state information concerning suspended processes must be accessible to the OS.

3. **Ready/Suspend** → **Ready**:

- When there are no ready processes in main memory, the OS will need to bring one in to continue execution.
- In addition, it might be the case that a process in the Ready/Suspend state has higher priority than any of the processes in the Ready state.
- In that case, the OS designer may dictate that it is more important to get at the higher-priority process than to minimize swapping.



Process State Transition Diagram with Suspend States

4. Ready → Ready/Suspend:

- Normally, the OS would prefer to suspend a blocked process rather than a ready one, because the ready process can now be executed, whereas the blocked process is taking up main memory space and cannot be executed.
- However, it may be necessary to suspend a ready process if that is the only way to free up a sufficiently large block of main memory.
- Also, the OS may choose to suspend a lower-priority ready process rather than a higher priority blocked process if it believes that the blocked process will be ready soon

5. New → Ready: A newly created process is placed in main memory and added to the Ready queue when sufficient memory is available.

6. New → Ready/Suspend: The OS creates the process and its PCB, but places it on disk because main memory is insufficient. It can be brought into memory later.

7. Blocked/Suspend → Blocked: A blocked process is brought back into main memory if it has high priority and the OS expects its awaited event to occur soon.

8. Running → Ready/Suspend: A running process may be preempted and moved directly to disk to free memory for a higher-priority process that has just become ready.

9. Any State → Exit: Although processes normally terminate while running, the OS or parent process may terminate a process in any state.

Thread

Topics to be covered

- 01 - Overview
- 02 - Multicore Programming
- 03 - Multithreading Models
- 04 - Thread Libraries
- 05 - Implicit Threading
- 06 - Threading Issues
- 07 - Operating System Examples

Threads

Single-Threaded Program

		<code>int x;</code>
		<code>x = 20;</code>
		<code>int y;</code>
Time		<code>y = 50;</code>
		<code>int sum;</code>
	↓	<code>sum = x + y;</code>

Threads

Multi-Threaded Program

Time

```
| int x;  
| x = 20;  
| int y;  
| y = 50;  
| int sum;  
↓ sum = x + y;
```

Time

```
| int a;  
| a = 3;  
| int b;  
| b = 5;  
| int product;  
↓ product = a * b;
```

Parallel Execution

Threads

Multi-Threaded Program

The diagram illustrates the execution of two code snippets over time. On the left, a vertical arrow labeled "Time" points downwards, indicating the flow of execution. The first code snippet consists of the following lines:

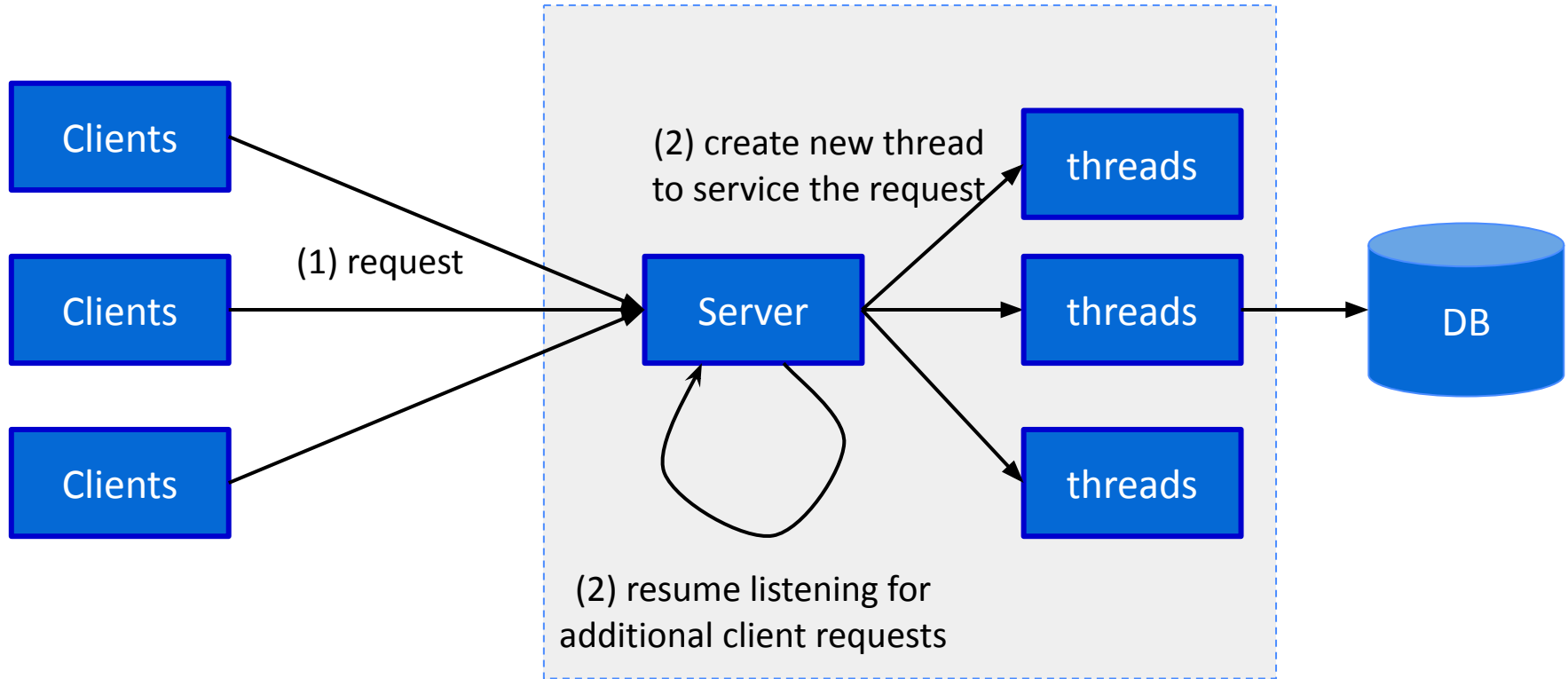
```
int x;  
  
x = 20;  
int y;  
  
y = 50;  
  
int sum;  
sum = x + y;
```

The second code snippet, located to the right, consists of the following lines:

```
int a;  
a = 3;  
  
int b;  
  
b = 5;  
int product;  
  
product = a * b;
```

Vertical lines separate the two snippets. At the bottom, a blue arrow points to the text "Concurrent But Not Parallel Execution", indicating that the two snippets are executed at the same time but on different processors, and their execution is not interleaved.

Multithreaded Server Architecture



Benefits

- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – process can take advantage of multiprocessor architectures

Multicore Programming

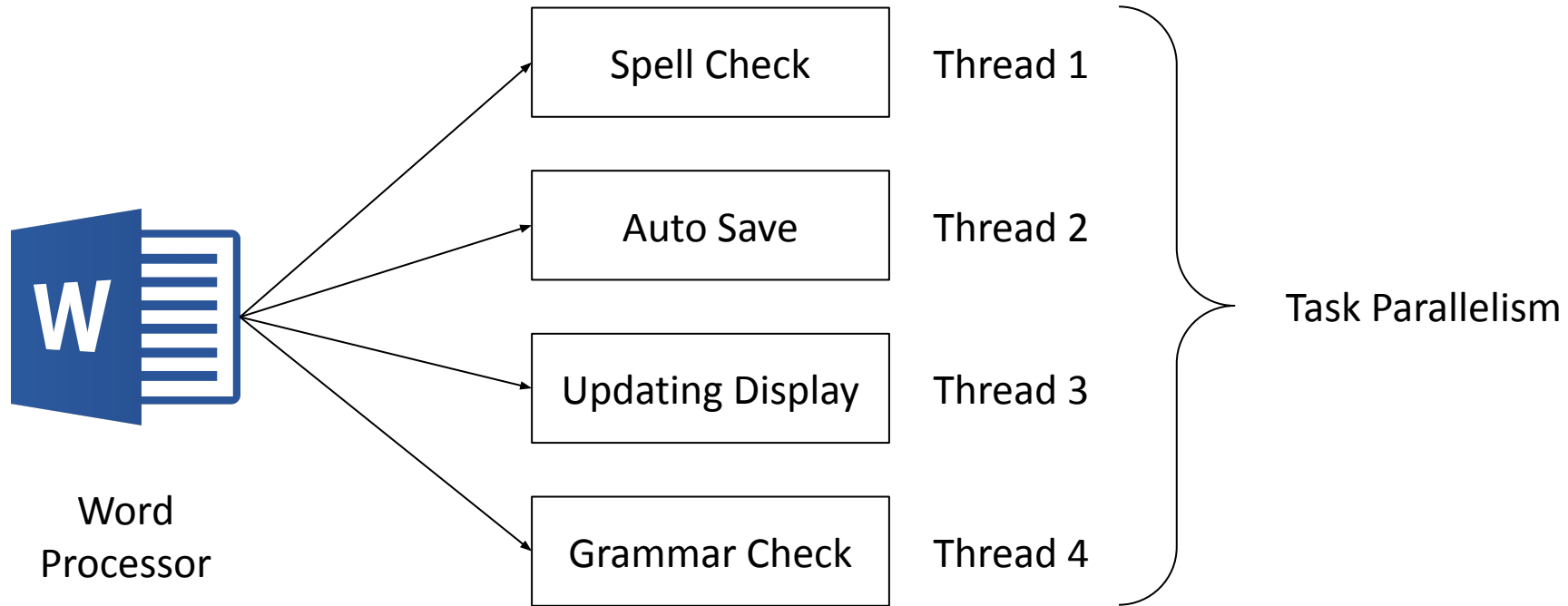
- **Multicore** or **Multiprocessor** systems putting pressure on programmers, challenges include:
 - Dividing activities
 - Load Balance across Threads
 - Data splitting
 - Data dependency
 - Testing and debugging
- Parallelism implies a system can perform more than one task simultaneously
- Concurrency supports more than one task making progress
 - Single processor / core, scheduler providing concurrency

Multicore Programming

- Types of parallelism
 - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
 - **Task parallelism** – distributes threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
- CPUs have cores as well as ***hardware threads***
- Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core

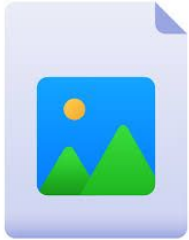


Task Parallelism



Data Parallelism

Apply Some Filter
/ Remove Noise



Large Size Image File
(1 million x 1 million
pixels)



Photoshop
Application



Thread 1

Thread 2

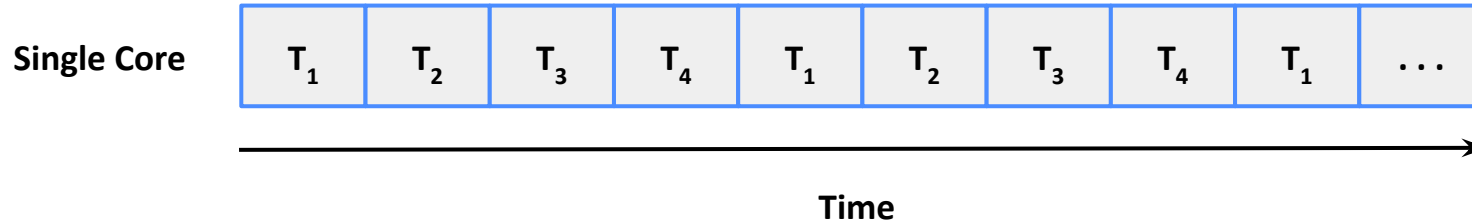
Thread 3

Thread 4

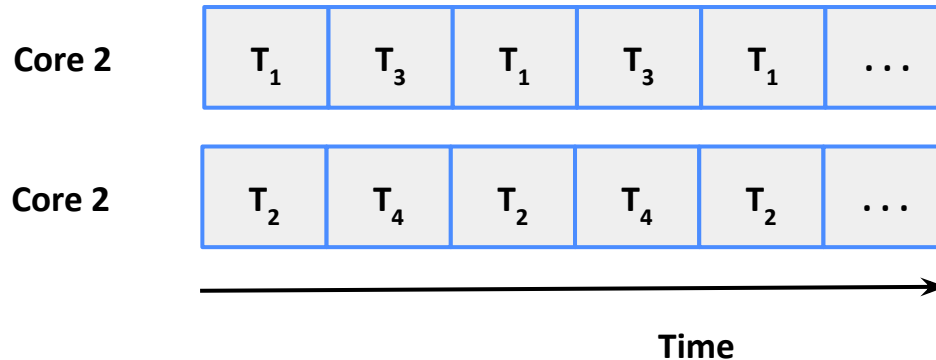
Data Parallelism

Concurrency vs. Parallelism

- Concurrent execution on single-core system:

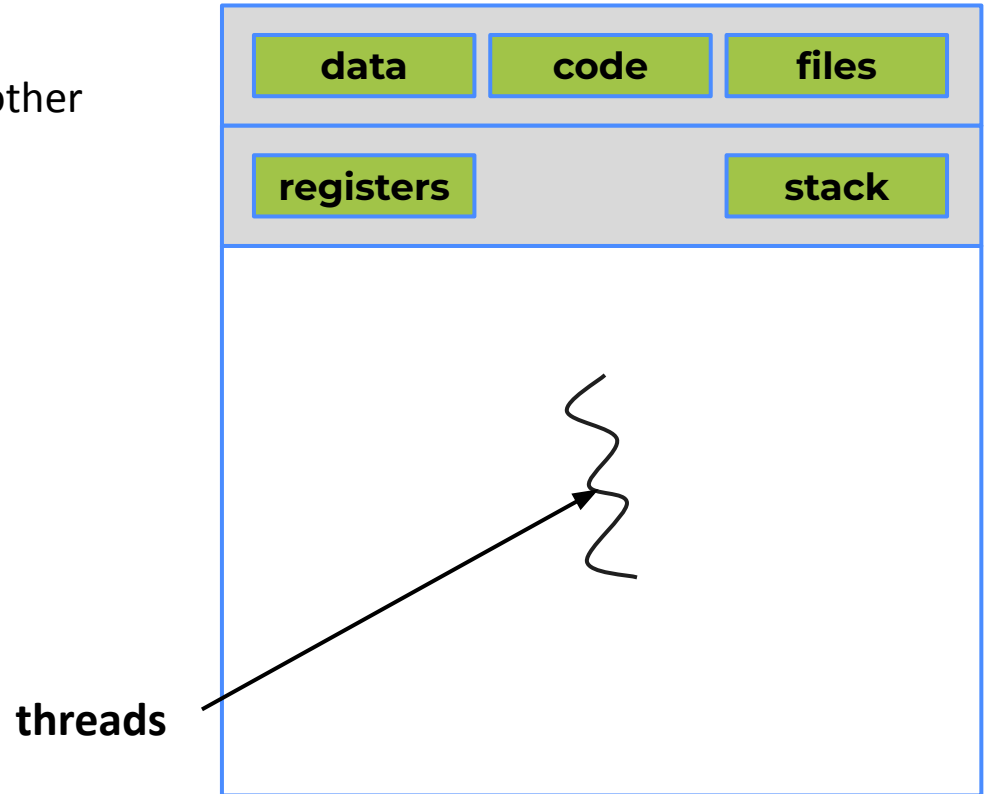


- Parallelism on a multi-core system:



(Single Threaded Process) / Child Processes

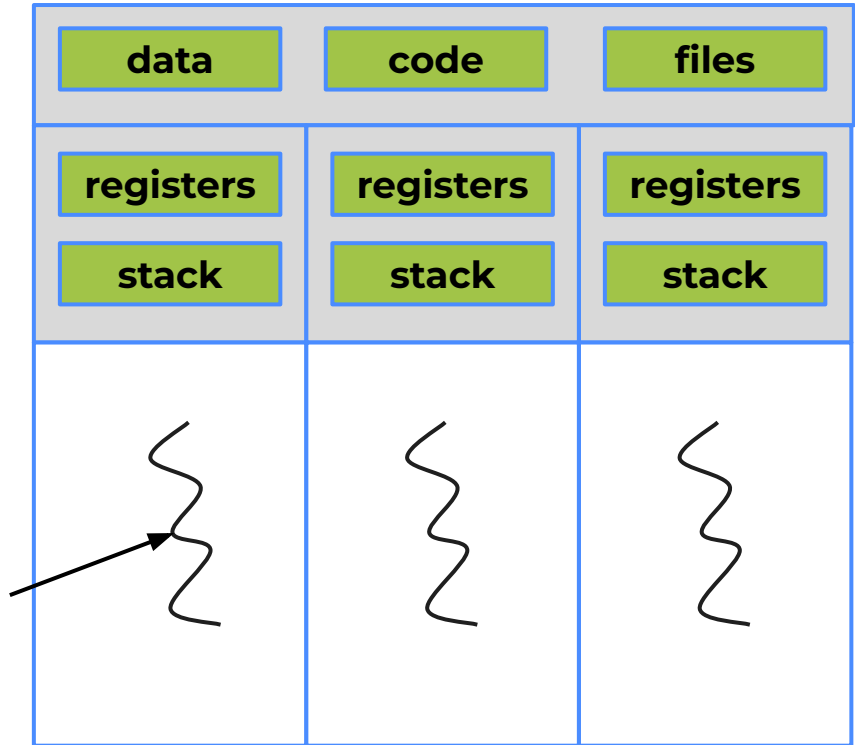
- Separate streams of execution
- Each process isolated from the other
- Process state contains
 - Process ID
 - Environment
 - Working directory.
 - Program instructions
 - Registers
 - Stack
 - Heap
 - File descriptors
- Created by the OS using `fork()`
 - **Significant overheads**



Multithreaded Process

- Separate streams of execution within a single process
- Threads in a process not isolated from each other
- Each thread state (thread control block) contains
 - Registers (including EIP, ESP)
 - stack

threads



Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial/sequential portion
- N processing cores

$$\text{Speedup} \leq 1 / (S + ((1 - S) / N))$$

- That is, if application is 75% parallel / 25% serial or sequential, moving from 1 to 2 cores results in speedup of **1.6 times** (for N = 4)
- As N approaches infinity, speedup approaches 1 / S
Serial portion of an application has disproportionate effect on performance gained by adding additional cores
- But does the law take into account contemporary multicore systems?

Why Threads

- Lightweight
- Efficient communication between entities
- Efficient context switching

Platform	fork			pthread_create()		
	real	user	sys	real	user	sys
Intel 2.6 GHz Xeon E5-2670 (16 cores/node)	8.1	0.1	2.9	0.9	0.2	0.3
Intel 2.8 GHz Xeon 5660 (12 cores/node)	4.4	0.4	4.3	0.7	0.2	0.5
AMD 2.3 GHz Opteron (16 cores/node)	12.5	1.0	12.5	1.2	0.2	1.3
AMD 2.4 GHz Opteron (8 cores/node)	17.6	2.2	15.7	1.4	0.3	1.3
IBM 4.0 GHz POWER6 (8 cpus/node)	9.5	0.6	8.8	1.6	0.1	0.4
IBM 1.9 GHz POWER5 p5-575 (8 cpus/node)	64.2	30.7	27.6	1.7	0.6	1.1
IBM 1.5 GHz POWER4 (8 cpus/node)	104.5	48.6	47.2	2.1	1.0	1.5
INTEL 2.4 GHz Xeon (2 cpus/node)	54.9	1.5	20.8	1.6	0.7	0.9
INTEL 1.4 GHz Itanium2 (4 cpus/node)	54.5	1.1	22.2	2.0	1.2	0.6

Cost of creating 50,000 processes / threads
https://hpc-tutorials.llnl.gov/posix/why_pthreads/



Threads vs Processes

Threads

- A thread has no data segment or heap
- A thread cannot live on its own. It needs to be attached to a process
- There can be more than one thread in a process.
- Each thread has its own stack
- If a thread dies, its stack is reclaimed
- There is no system call involved

Processes

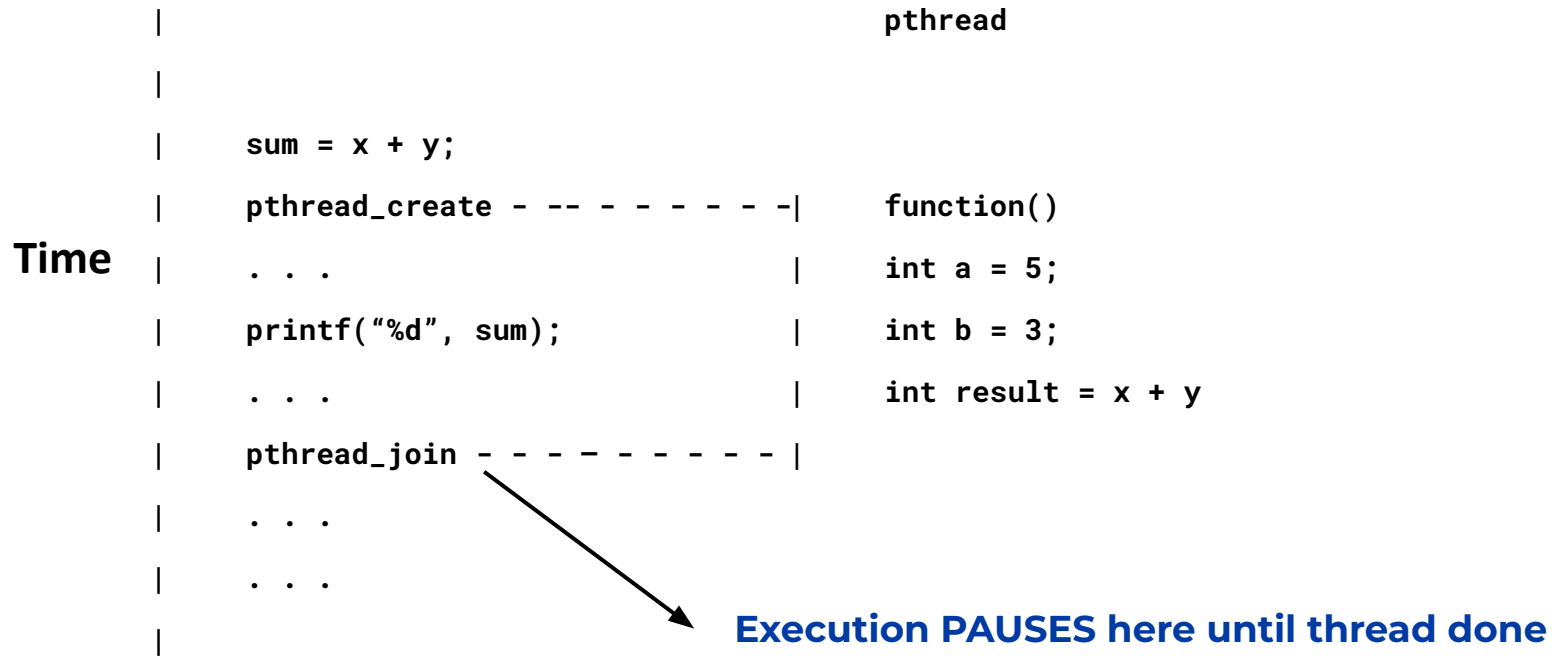
- A process has code, heap, stack, other segments
- A process has at-least one thread.
- Threads within a process share the same I/O, code, files.
- If a process dies, all threads die.
- System call involved in Process

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- Specification, not implementation
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

pthread library

Multi-Threaded Program



Parallel Execution

pthread library

```
#include <stdio.h>
#include <pthread.h>          //POSIX Thread Library

void *computation();          // The function should return a void*

int main(){
    pthread_t thread1;        // This is used to create and manage our thread

    //pthread_create is going to run a function in a separate thread
    //arg1 - pointer to the pthread_t variable, arg2 - to set the attributes
    //of the thread, arg3 - the function itself, arg4 - to pass args to the function computation
    pthread_create(&thread1, NULL, computation, NULL);

    // To join the thread back with the main thread execution.
    // arg1 - the thread that you are joining with the main thread execution, arg2 -
    // used to work with the return value from that function
    pthread_join(thread1, NULL);
    return 0;
}

void *computation(){
    printf("Computation\n");
    return NULL;
}
```

Output:

```
$ gcc threadexample1.c
$ ./a.out
Computation
$
```



pthread library

To Pass an argument to the functions

```
#include <stdio.h>
#include <pthread.h>          //POSIX Thread Library

void *computation(void *add);    // The function should return a void*

int main(){
    pthread_t thread1;          // This is used to create and manage our thread

    //pthread_create is going to run a function in a separate thread
    //arg1 – pointer to the pthread_t variable, arg2 – to set the attributes of the thread, arg3 – the function itself, arg4 – to pass args to the
    function computation
    long value1 = 1;
    pthread_create(&thread1, NULL, computation, (void*) &value1); //arg4 has to be a void pointer to pass in a value via a pointer or pass
    a struct via a pointer to that struct

    // To join the thread back with the main thread execution.
    // arg1 – the thread that you are joining with the main thread execution, arg2 –
    used to work with the return value from that function
    pthread_join(thread1, NULL);
    return 0;

}

void *computation(void *add){
    long *add_num = (long *) (add);
    printf("Add: %ld\n", *add_num);
    return NULL;
}
```

```
Output:
$ gcc threadexample2.c
$ ./a.out
Add: 1
$
```



pthread library

To create multiple threads

```
#include <stdio.h>
#include <pthread.h>          //POSIX Thread Library

void *computation(void *add);    // The function should return a void*

int main(){
    pthread_t thread1;
    pthread_t thread2;

    long value1 = 1;
    long value2 = 5;

    pthread_create(&thread1, NULL, computation, (void*) &value1);
    pthread_create(&thread2, NULL, computation, (void*) &value2);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}

void *computation(void *add){
    Long *add_num = (long *) (add);
    printf("Add: %ld\n", *add_num);
    return NULL;
}
```

Output:

```
$ gcc threadexample3.c
$ ./a.out
Add: 1
Add: 5
$
```



pthread library

- Create a thread in a process

```
int pthread_create(pthread_t *thread,  
    const pthread_attr_t *attr,  
    void *(*start_routine) (void *),  
    void *arg);
```

Thread identifier (TID) much like

Pointer to a function, which starts execution in a different thread

Arguments to the function

- Destroying a thread

```
void pthread_exit(void *retval);
```

Exit value of the thread

Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```



Pthreads Example (Contd.)

```
/* get the default attributes */
pthread_attr_t attr;
pthread_attr_init(&attr);
/* create the thread */
pthread_create(&tid,&attr,runner,argv[1]);
/* wait for the thread to exit */
pthread_join(tid,NULL);

printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```



pthread library

- Join : Wait for a specific thread to complete

```
int pthread_join(pthread_t thread, void **retval);
```

TID of the thread to wait for

Exit status of the thread

what is the difference with wait()?

Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Thread Libraries

- Thread library provides programmer with API for creating and managing threads
- Two primary ways of implementing
- Library entirely in user space
- Kernel-level library supported by the OS

Other thread libraries

- Windows threads
- Boost (in C++)
- LinuxThreads
- etc.

Who manages threads?

User-Level Threads

- Thread management done by user level thread library.
- Kernel knows nothing about the threads.
- Typically Fast
- Context switching is faster
- Three primary thread libraries:
 - POSIX Pthreads
 - Windows threads
 - Java threads

Kernel-Level threads

- Threads directly supported by the kernel. (managed by Syscall)
- Known as light weight processes
- Slower than user-level
- Context switching is slower
- Examples – virtually all general purpose operating systems, including:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X

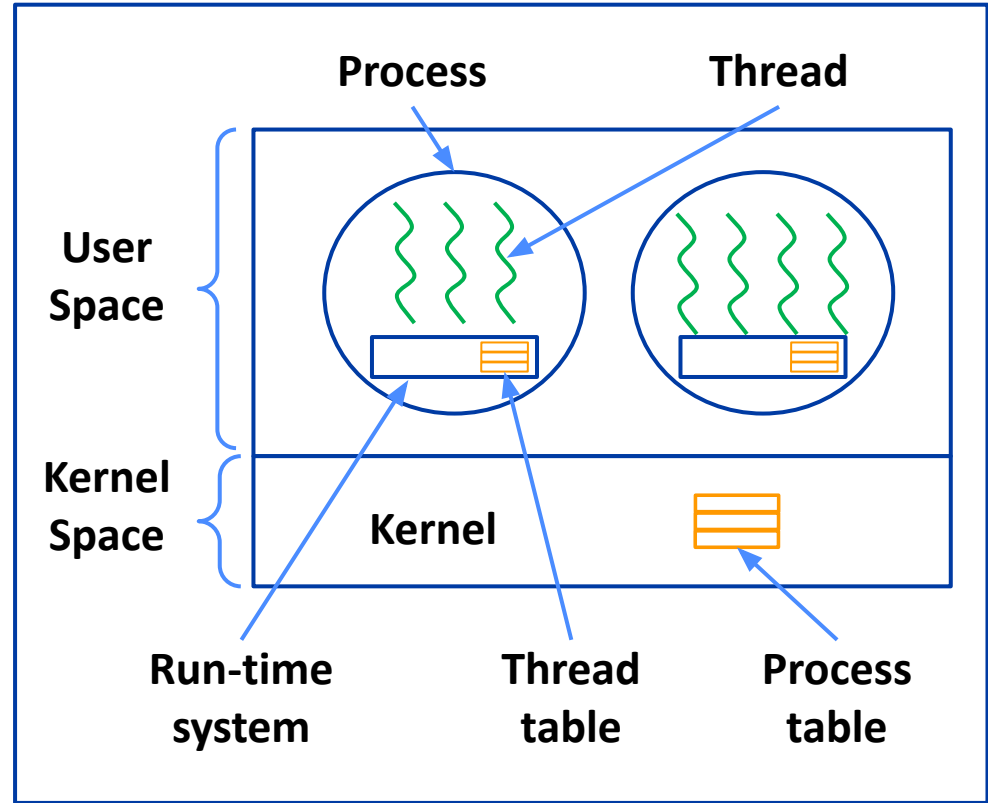
User level threads

Advantages

- Fast (really lightweight)
(no system call to manage threads. The thread library does everything).
- Can be implemented on an OS that does not support threading.
- Switching is fast. No, switch from user to protected mode.

Disadvantages

- Scheduling can be an issue. (Consider, one thread that is blocked on an IO and another runnable.)
- Lack of coordination between kernel and threads. (A process with 1000 threads competes for a timeslice with a process having just 1 thread.)
- Requires non-blocking system calls. (If one thread invokes a system call, all threads need to wait)



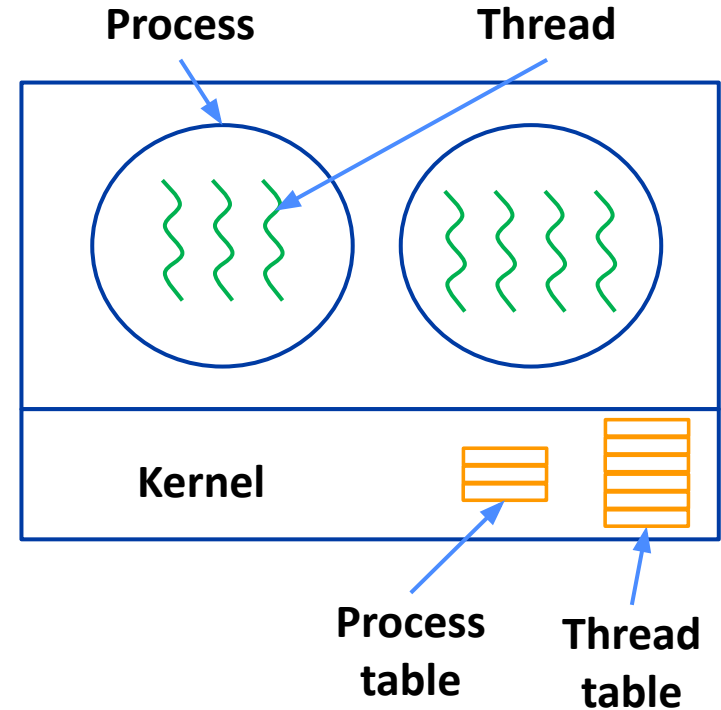
Kernel level threads

Advantages

- Scheduler can decide to give more time to a process having large number of threads than process having small number of threads.
- Kernel-level threads are especially good for applications that frequently block.

Disadvantages

- The kernel-level threads are slow (they involve kernel invocations.)
- Overheads in the kernel. (Since kernel must manage and schedule threads as well as processes. It requires a full thread control block (TCB) for each thread to maintain information about threads.)

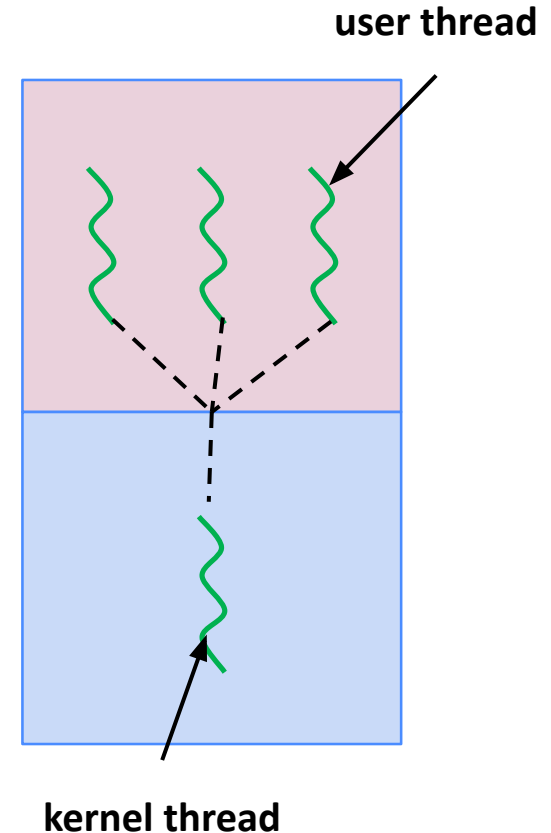


Thread Models

- Many-to-one model
- One-to-one model
- Many-to-many model

Many-to-One Model

- Many user level threads map to a single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
 - Solaris Green Threads
 - GNU Portable Threads



Many-to-One Model

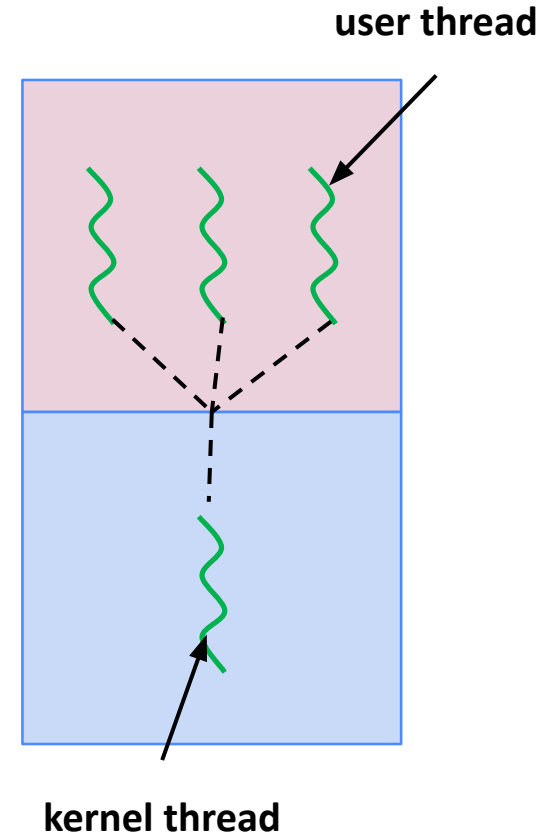
- Many user level threads map to a single kernel thread

Advantages

- Fast. No system calls to manage threads.
- No mode change for switching threads

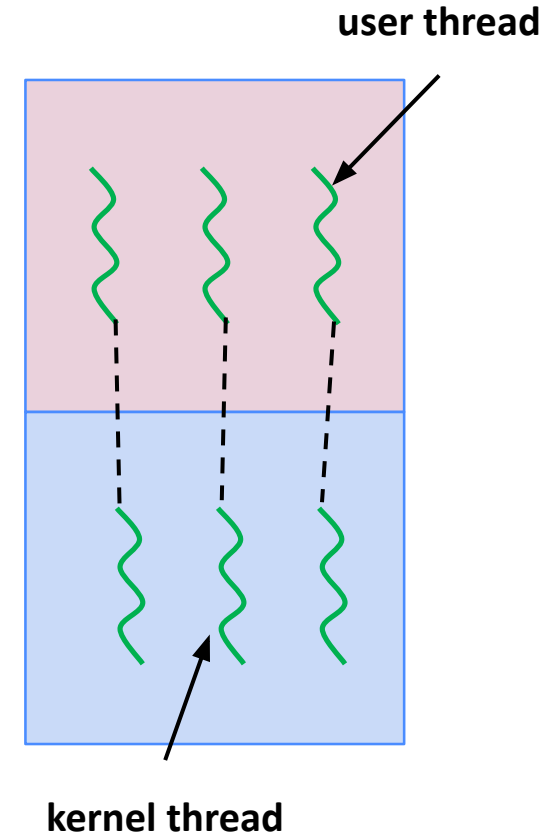
Disadvantages

- No parallel execution of threads. All threads block when one has a system call.
- Not suited for multi-processor systems.



One-to-One Model

- Each user thread associated with one kernel thread.
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
 - Windows
 - Linux
 - Solaris 9 and later



One-to-One Model

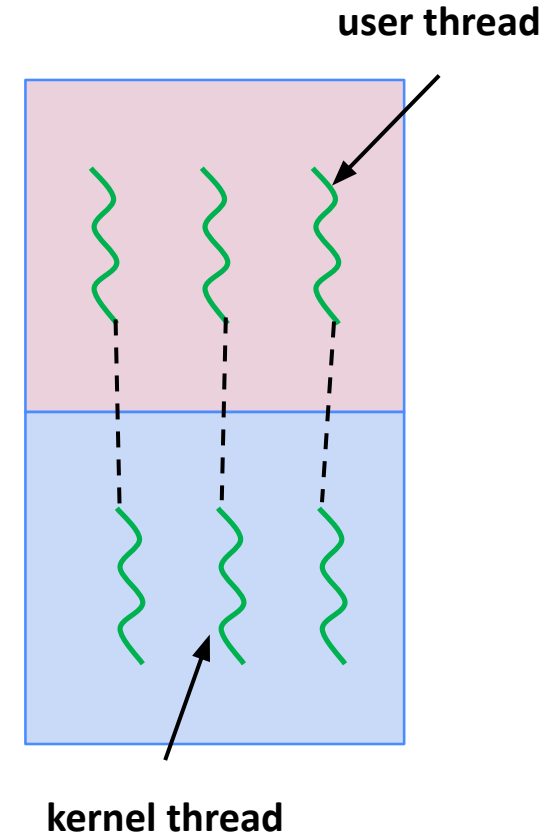
- Each user thread associated with one kernel thread.

Advantages

- Better suited for multiprocessor environments.
- When one thread blocks, the other threads can continue to execute.

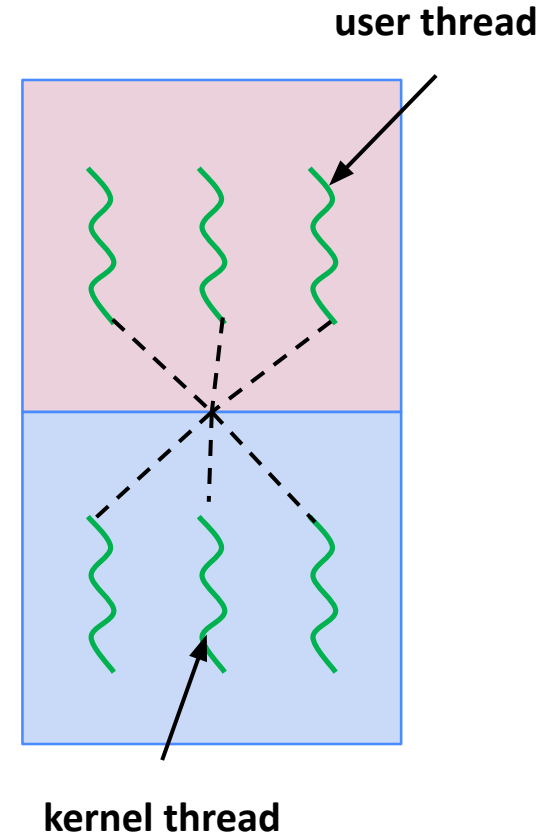
Disadvantages

- Expensive. Kernel is involved



Many-to-Many Model

- Many user threads mapped to many kernel threads
 - Supported by some unix and windows versions
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the **ThreadFiber** package



Many-to-Many Model

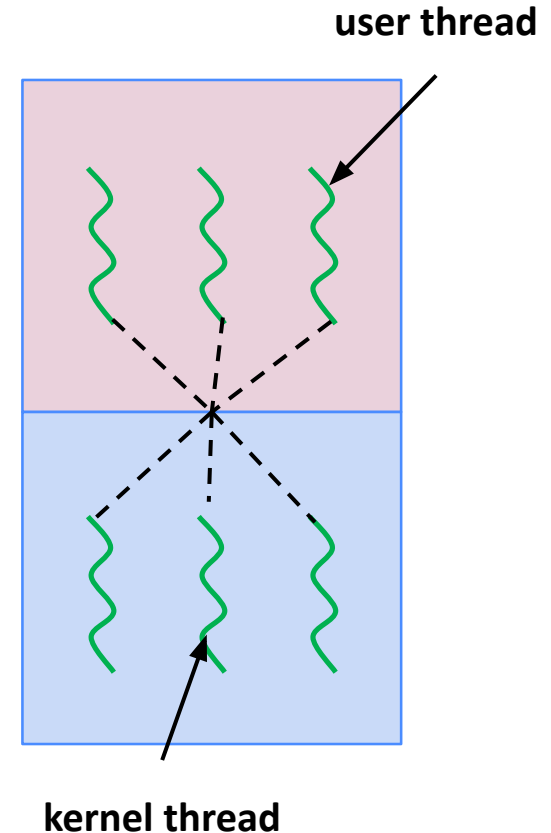
- Many user threads mapped to many kernel threads
 - Supported by some unix and windows versions

Advantages

- Flexible
- OS creates kernel threads as required
- Process creates user threads as needed

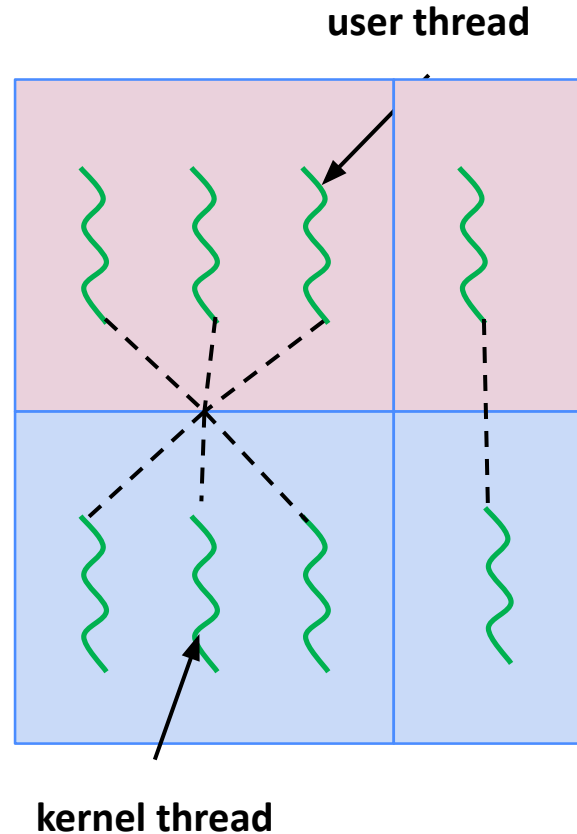
Disadvantages

- Complex
- Double management



Two-level Model

- Similar to M:M, except that it allows a user thread to be bound to kernel thread
- Examples
 - IRIX
 - HP-UX
 - Tru64 UNIX
 - Solaris 8 and earlier



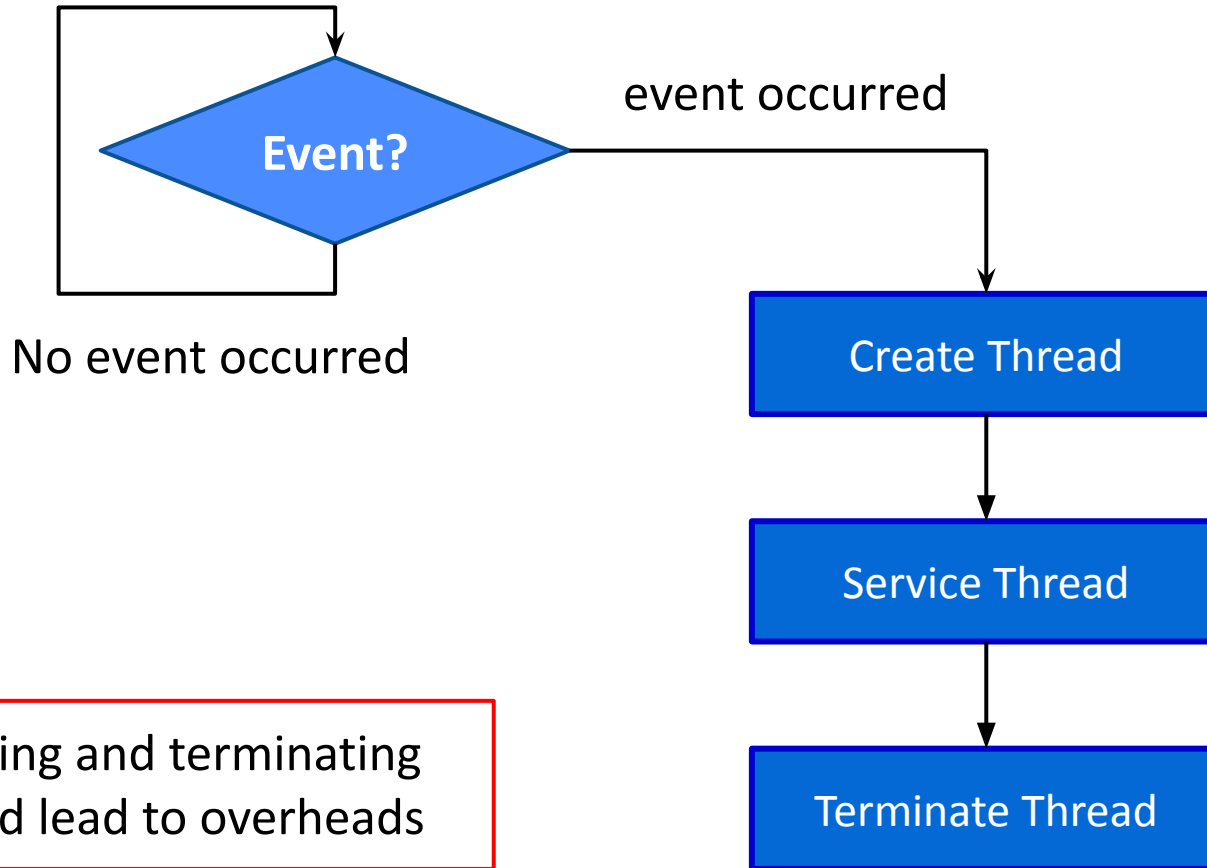
Threading issues

- Semantics of fork() and exec() system calls
- Signal handling
 - Synchronous and asynchronous
- Thread cancellation of target thread
 - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations

Threading issues

- What happens when a thread invokes fork?
 - Duplicate all threads?
 - Not easily done... other threads may be running or blocked in a system call
 - Duplicate only the caller thread?
 - More feasible.
- Segmentation fault in a thread. Should only the thread terminate or the entire process?

Typical usage of threads



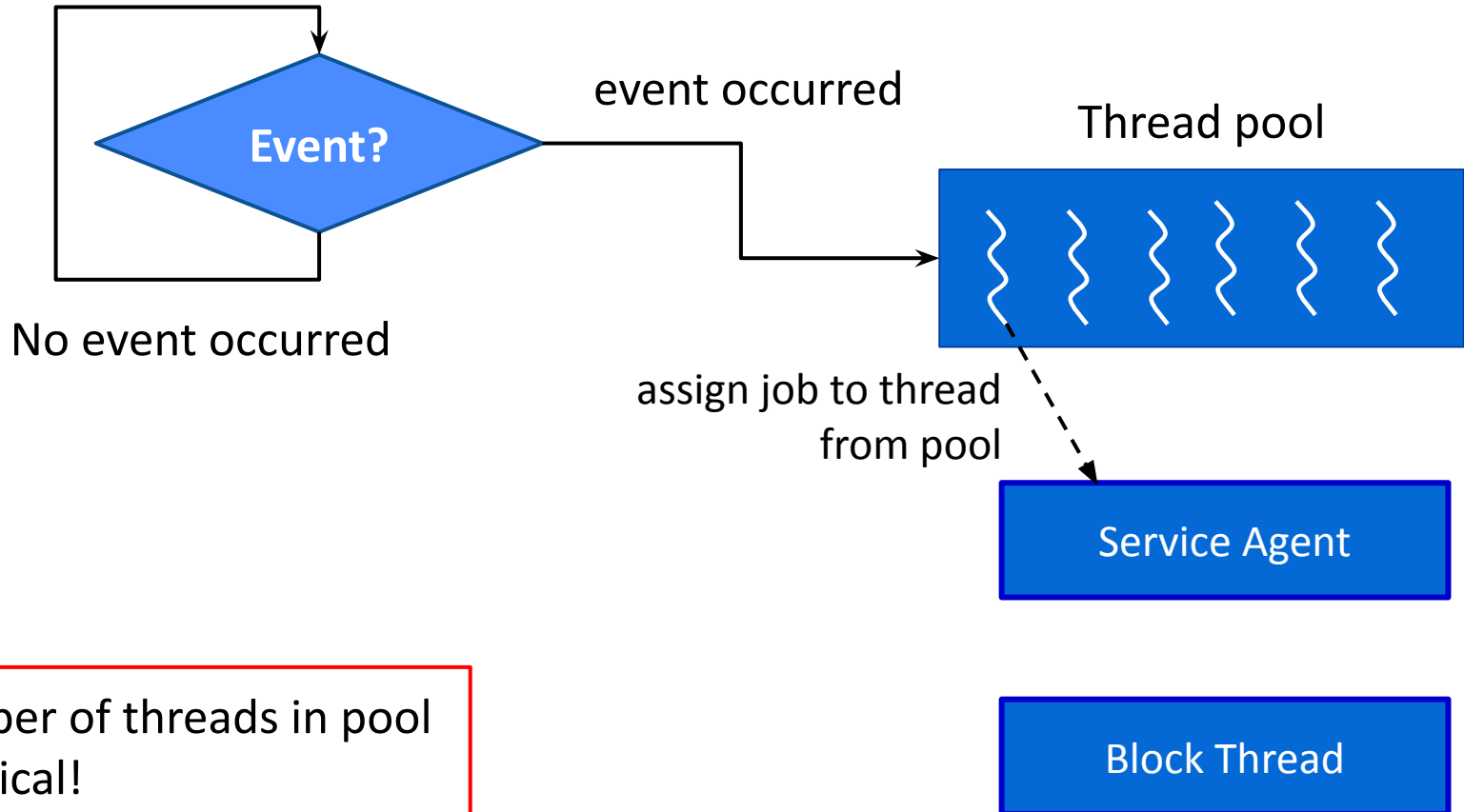
Creating and terminating thread lead to overheads

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task
 - i.e. Tasks could be scheduled to run periodically
- Windows API supports thread pools:

```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /*  
     * this function runs as a separate thread.  
     */  
}
```

Thread pools



Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is target thread
- Two general approaches:
 - Asynchronous cancellation terminates the target thread immediately
 - Deferred cancellation allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;  
  
/* create the thread */  
pthread_create(&tid, 0, worker, NULL);  
  
. . .  
  
/* cancel the thread */  
pthread_cancel(tid);
```



Thread Scheduling

- Distinction between user-level and kernel-level threads
- When threads supported, threads scheduled, not processes
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
 - Known as process-contention scope (PCS) since scheduling competition is within the process
 - Typically done via priority set by programmer
- Kernel thread scheduled onto available CPU is system-contention scope (SCS) – competition among all threads in system

Homework

- Write a multi-threaded program to perform matrix multiplication.
 - The input matrices are of dimension 1000×1000
 - Tabulate time taken vs number of threads used.

Next Class We Will Talk About

- Cooperating processes
- IPC overview, Inter-Process Communication (IPC)
- IPC: shared memory, pipes, message passing
- Client–Server communication

Happy Learning !

We're Done.



Questions?

quibunnies.com

